

Kubernetes Persistent Storage in Amazon EKS — Complete Step-by-Step Guide

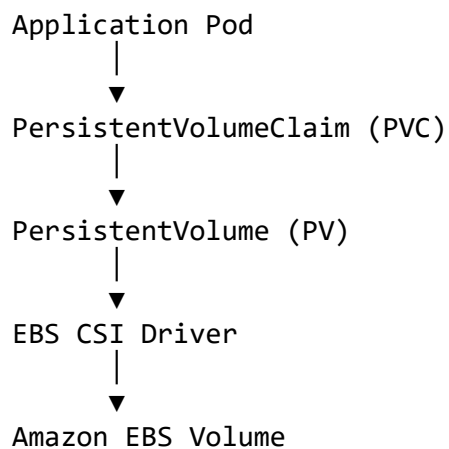
This document explains:

1. Prerequisites
2. Helm Installation
3. EBS CSI Driver Setup
4. StorageClass Configuration
5. PVC Creation
6. Deployment Using PVC
7. Data Persistence Testing
8. Service Exposure
9. Cleanup
10. Interview Questions

This setup is production-ready for:

- Amazon EKS
 - Kubernetes
-

Architecture Overview



1 Prerequisites

Before starting, ensure:

Requirement	Description
AWS CLI	Installed and configured
kubectl	Installed
eksctl	Installed
Helm	Installed
EKS Cluster	Running
Worker Nodes	Ready state

2 Verify Cluster Access

`kubectl get nodes`

Expected:

NAME	STATUS	ROLES	AGE	VERSION
worker-node-1	Ready	<none>	10d	v1.30

3 Install Helm

Install Helm on Linux

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |
bash
```

Verify Helm Installation

`helm version`

Expected:

```
version.BuildInfo{Version:"v3.x.x"}
```

4 Configure EBS CSI Driver in EKS

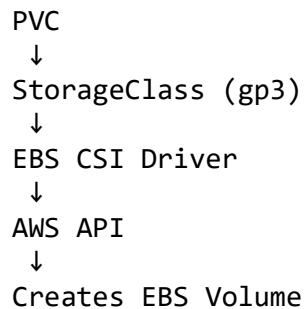
The EBS CSI Driver is required for:

- Dynamic EBS provisioning
- Persistent Volumes in EKS
- Stateful workloads

Without this:

- PVC may remain Pending
- Kubernetes cannot dynamically create EBS volumes

EBS CSI Driver Flow



5 Enable IAM OIDC Provider

```
eksctl utils associate-iam-oidc-provider \
  --region <your-region> \
  --cluster <your-cluster-name> \
  --approve
```

Example:

```
eksctl utils associate-iam-oidc-provider \
  --region ap-south-1 \
  --cluster demo-eks-cluster \
  --approve
```

Why OIDC is Required

OIDC enables:

- IAM Roles for Service Accounts (IRSA)
 - Secure AWS authentication from Kubernetes pods
-

6 Create IAM Service Account

```
eksctl create iamserviceaccount \  
  --name ebs-csi-controller-sa \  
  --namespace kube-system \  
  --cluster <your-cluster-name> \  
  --attach-policy-arn arn:aws:iam::aws:policy/service-  
role/AmazonEBSCSIDriverPolicy \  
  --approve \  
  --role-name AmazonEKS_EBS_CSI_DriverRole
```

Example:

```
eksctl create iamserviceaccount \  
  --name ebs-csi-controller-sa \  
  --namespace kube-system \  
  --cluster demo-eks-cluster \  
  --attach-policy-arn arn:aws:iam::aws:policy/service-  
role/AmazonEBSCSIDriverPolicy \  
  --approve \  
  --role-name AmazonEKS_EBS_CSI_DriverRole
```

What This Creates

- Kubernetes Service Account
- IAM Role
- IAM Policy Attachment

Policy Used:

AmazonEBSCSIDriverPolicy

7 Install EBS CSI Driver Using Helm

Add Helm Repository

```
helm repo add aws-ebs-csi-driver https://kubernetes-sigs.github.io/aws-ebs-csi-driver
```

Update repository:

```
helm repo update
```

Install CSI Driver

```
helm install aws-ebs-csi-driver \
  aws-ebs-csi-driver/aws-ebs-csi-driver \
  --namespace kube-system \
  --set controller.serviceAccount.create=false \
  --set controller.serviceAccount.name=ebs-csi-controller-sa
```

8 Verify CSI Driver Installation

```
kubectl get pods -n kube-system | grep ebs
```

Expected:

ebs-csi-controller-xxxxx	Running
ebs-csi-node-xxxxx	Running

9 Verify StorageClass

```
kubectl get storageclass
```

Expected:

NAME	PROVISIONER
gp3 (default)	ebs.csi.aws.com

Important:

Use:

```
ebs.csi.aws.com
```

Old provisioner:

kubernetes.io/aws-ebs

is deprecated.

10 Create gp3 StorageClass

gp3-storageclass.yaml

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3
```

```
provisioner: ebs.csi.aws.com
```

```
volumeBindingMode: WaitForFirstConsumer
```

```
parameters:
  type: gp3
  fsType: ext4
```

```
allowVolumeExpansion: true
```

Apply:

```
kubectl apply -f gp3-storageclass.yaml
```

1 1 Create Namespace

```
kubectl create namespace storage-demo
```

Verify:

```
kubectl get ns
```

1 2 Create PersistentVolumeClaim (PVC)

pvc.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
```

```

name: demo-pvc
namespace: storage-demo

spec:
  accessModes:
    - ReadWriteOnce

  storageClassName: gp3

resources:
  requests:
    storage: 5Gi
  
```

Apply PVC

```
kubectl apply -f pvc.yaml
```

Verify PVC

```
kubectl get pvc -n storage-demo
```

Expected:

NAME	STATUS	VOLUME
demo-pvc	Bound	pvc-xxxxxx

Verify PV

```
kubectl get pv
```

1 3 Create Deployment Using PVC

deployment.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-pv-demo
  namespace: storage-demo
  
```

```
spec:
  replicas: 1

  selector:
    matchLabels:
      app: nginx-pv-demo

  template:
    metadata:
      labels:
        app: nginx-pv-demo

    spec:
      containers:
        - name: nginx
          image: nginx:1.27

          ports:
            - containerPort: 80

          volumeMounts:
            - name: demo-storage
              mountPath: /usr/share/nginx/html

      volumes:
        - name: demo-storage
          persistentVolumeClaim:
            claimName: demo-pvc
```

Apply Deployment

```
kubectl apply -f deployment.yaml
```

Verify Pod

```
kubectl get pods -n storage-demo
```

Expected:

```
nginx-pv-demo-xxxxx    Running
```

1 4 Write Data Into Persistent Storage

Get pod name:

```
kubectl get pods -n storage-demo
```

Enter pod:

```
kubectl exec -it <POD_NAME> -n storage-demo -- bash
```

Inside container:

```
cd /usr/share/nginx/html
```

```
echo "Hello from Persistent Storage" > index.html
```

```
cat index.html
```

Expected:

Hello from Persistent Storage

Exit:

```
exit
```

1 5 Test Persistence

Delete pod:

```
kubectl delete pod <POD_NAME> -n storage-demo
```

New pod will automatically be created.

Verify:

```
kubectl get pods -n storage-demo
```

Verify Persistent Data

Enter new pod:

```
kubectl exec -it <NEW_POD_NAME> -n storage-demo -- bash
```

Check file:

```
cd /usr/share/nginx/html
```

```
cat index.html
```

Expected:

Hello from Persistent Storage

SUCCESS 

Data persists because PVC remains attached.

1 6 Expose Application (Optional)

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-pv-service
  namespace: storage-demo

spec:
  type: NodePort

  selector:
    app: nginx-pv-demo

  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

Apply Service

```
kubectl apply -f service.yaml
```

Verify:

```
kubectl get svc -n storage-demo
```

Access Application

http://<NODE-IP>:30080

1 7 Cleanup

Delete service:

```
kubectl delete -f service.yaml
```

Delete deployment:

```
kubectl delete -f deployment.yaml
```

Delete PVC:

```
kubectl delete -f pvc.yaml
```

Delete namespace:

```
kubectl delete namespace storage-demo
```

Important Concepts

Component	Purpose
Pod	Runs container
PVC	Requests storage
PV	Actual storage
StorageClass	Dynamically provisions storage
EBS CSI Driver	Creates and manages EBS volumes

Complete EKS Dynamic Provisioning Flow

Create PVC



StorageClass selected



EBS CSI Driver triggered



AWS API called



EBS Volume created
↓
PersistentVolume created
↓
PVC Bound
↓
Pod mounts storage

Common Issues

PVC Stuck in Pending

Possible reasons:

- CSI Driver not installed
 - No StorageClass
 - Wrong provisioner
 - IAM permission issue
-

Useful Commands

```
kubectl get pvc -A  
  
kubectl get pv  
  
kubectl describe pvc demo-pvc -n storage-demo  
  
kubectl describe pv  
  
kubectl get storageclass  
  
kubectl get pods -n kube-system | grep ebs  
  
helm list -A
```

Interview Questions

Difference Between PV and PVC

PV

PVC

PV	PVC
Actual storage	Storage request
Cluster resource	Namespace resource
Created dynamically/manually	Created by user

What happens if pod is deleted?

- Pod deleted ❌
- PVC remains ✅
- Data remains ✅

Why is EBS CSI Driver needed?

Because Kubernetes cannot directly create EBS volumes.

CSI Driver handles:

- EBS creation
- Volume attachment
- Volume mounting
- Dynamic provisioning

Final Summary

Production-ready EKS persistent storage setup includes:

✅ Helm
 ✅ EBS CSI Driver
 ✅ OIDC Provider
 ✅ IRSA
 ✅ gp3 StorageClass
 ✅ Dynamic PVC Provisioning
 ✅ Persistent Data Across Pod Restarts